

Studi Kasus

Amela Luna Andrealova

25/566386/TK/63891

Problem 1

- Diketahui :
- > Kotak 1 : n barang, berat $2\text{kg}/\text{barang}$
 - > Kotak 2 : m barang, berat $1\text{kg}/\text{barang}$
 - > Kotak 3 : kosong
 - > Kapasitas robot maksimal $k\text{ kg}$
 - > Tidak boleh campur barang 2kg dan 1kg dalam satu kali angkutan.
 - > 1 ambil = 1 step, 1 taruh = 1 step

Ditanya = Langkah minimum agar 3 kotak punya jumlah barang dan total berat sama, atau -1 jika penyeimbangan tidak mungkin dicapai

Jawaban = Misalkan kotak i berisi a_i barang 2kg dan b_i barang 1kg .

- ↳ Jumlah barang = $a_i + b_i$
- ↳ Berat total = $2a_i + b_i$

Jika jumlah dan berat sama di ketiga kotak, maka

$$\begin{aligned}\hookrightarrow \text{Berat total} - \text{jumlah barang} &= (2a_i + b_i) - (a_i + b_i) \\ &= 2a_i - a_i + b_i - b_i \\ &= a_i\end{aligned}$$

Jadi berat - jumlah = jumlah barang 2 kg

$$a_1 = a_2 = a_3 = \frac{n}{3}$$

karena $a_1 + a_2 + a_3 = n$, maka setiap kotak harus mempunyai tepat $n/3$ barang 2 kg dan otomatis $m/3$ barang 1 kg.

$$n \bmod 3 = 0 \quad \text{dan} \quad m \bmod 3 = 0$$

jika tidak 0, langsung -1

* misalkan $X = n/3$ dan $Y = m/3$, tidak boleh membawa barang 2kg dan 1 kg secara bersamaan, jadi langkahnya dihitung terpisah lalu dijumlah.

* Kapasitas robot dengan berat w

$$\text{Cap} = \frac{k}{w}$$

1 kali ambil barang bisa ambil banyak sekaligus asalkan berat barang $\leq$ kapasitasnya, tapi kalau muatan itu tujuannya ke 2 kotak yg berbeda maka butuh 2 kali taruh. Jadi setiap sekali jalan > 1 tujuan : 2 langkah (ambil, taruh)
 > 2 tujuan : 3 langkah (ambil, 2 taruh)

dengan $q = \frac{x}{cap}$, sisa $r = x \bmod cap$

$$steps(x, cap) = 4q + \begin{cases} 0 & r = 0 \\ 3 & 0 < r, 2r \leq cap \\ 4 & 2r > cap \end{cases}$$

↳ lebih optimal, menggabungkan sisa jadi 1 trip 2 tujuan kalau muat, jika tidak muat bisa pisah 1 tujuan lebih hemat daripada 2 trip 2 tujuan. Dan jika $x > 0$ tapi $cap = 0$ (k terlalu kecil) maka outputnya -1.

$$Total = steps\left(\frac{n}{3}, \left(\frac{k}{2}\right)\right) + steps\left(\frac{m}{3}, k\right)$$

* Syarat $n \% 3 = 0$, $m \% 3 = 0$ dan $cap \geq 1$ jika $x/y > 0$

Problem 2

- Diketahui :
- > Baterai kapasitas maksimum C awal terisi penuh.
 - > n misi berurutan, misi ke- i robot butuh energi sebesar x_i
 - > Sebelum tiap misi, boleh charge ($+R$, dibatasi maksimum C , tidak boleh overflow) atau tidak charge
 - > Maksimal K kali charging total selama seluruh misi.
 - > Misi hanya bisa dijalankan jika baterai $\geq$ kebutuhan misi
 - > Tidak dapat melewati misi untuk mengerjakan misi berikutnya, misi harus dikerjakan secara urut.

Ditanya = Jumlah maksimum misi yang dapat diselesaikan dengan strategi charging terbaik.

Jawaban =

karena semua x_i sudah diketahui di awal, kita bisa merencanakan sejak awal kapan sebaiknya charging dipakai. Timing kapan kita charge itu penting karena baterai dibatasi maksimum C . Pilihan di tiap misi hanya 2 (charge atau tidak charge) dan harus memaksimalkan sisa baterai di tiap titik agar peluang menyelesaikan misi berikutnya lebih besar.

Anggap $dp[i][j]$ = sisa baterai paling maksimal yang tersisa setelah berhasil ngerjain i misi pertama dengan j kali charge.

Transisi dari $dp[i][j]$ menuju misi ke- $(i+1)$ dengan kebutuhan $x = x[i+1]$

> Ga dicharge = jika $dp[i][j] \geq x$, maka
$$dp[i+1][j] = \max(dp[i+1][j], dp[i][j] - x).$$

Kalau baterai sekarang cukup, ya jalanin aja, sisa baterai dikurangi kebutuhan misi.

> Charge dulu (butuh $j+1 \leq k$)

↳ $terisi = \min(C, dp[i][j] + R)$. Jika $terisi \geq x$, maka $dp[i+1][j+1] = \max(dp[i+1][j+1], terisi - x)$.

Baterai jadi $\min(C, \text{sekarang} + R)$, biar ga kelebihan dari C , kalau itu cukup, jalanin misi, sisa dikurangi kebutuhan.

Jawaban akhirnya =

misi terjauh yang masih bisa dicapai lewat kombinasi (i, j) manapun.

Problem 3

Diketahui : > 8 loker (nomor 1-8), masing-masing loker i biasanya cuma bisa dibuka pakai kunci nomor i .

> Punya 1 kunci master bisa dipakai buka loker manapun sebagai langkah pertama.

> Di dalam loker ada tepat 1 kunci loker lain atau lokernya kosong.

- > Begitu dapat kunci, kunci tersebut harus digunakan untuk membuka loker sesuai nomor kunci.
- > Proses berhenti kalau kunci mengarah ke loker yang sudah pernah dibuka atau bernilai 0 (loker kosong).

Ditanya : Apakah ada pilihan loker awal untuk kunci master yang membuatku bisa membuka semua loker?

Jawaban : Misalkan tiap loker itu kayak titik dan kuncinya itu kayak panah nunjuk ke loker berikutnya. Jadi proses buka membuka loker itu ya cuma nyusurin rantai panah dari satu titik awal sampai mentok.

karena kunci master bisa dipake di loker manapun, ya udah tinggal coba aja ke-8 kemungkinan titik awal satu-satu, susurin rantainya danitung ada berapa loker yang kebuka. kalau ada satu aja titik awal yang

berhasil buka kedelapan-delapannya, jawabannya YES. kalau semua titik awal mentok sebelum sampe 8, ya NO.

Intinya

> kalau semua loker itu ternyata satu lingkaran besar yang saling nyambung ($1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow \dots \rightarrow 8 \rightarrow$ balik ke 1), mulai dari manapun pasti bisa keliling semua, YES.

> kalau loker kepecah jadi beberapa lingkaran / rantai kecil yang gak nyambung satu sama lain, ya gak bakal ada titik awal yang bisa nembus semuanya. NO.

Problem 4

Diketahui : > String s sepanjang n, hanya berisi karakter "(" dan ")"

> "(" = robot masuk ruangan

")" = robot keluar ruangan

> setiap ")" harus punya "(" yang mendahuluinya, tidak boleh

keluar dari ruangan yang belum dimasuki.

- > Robot tidak boleh keluar saat sedang tidak berada di dalam ruangan manapun
- > setelah semua perintah dijalankan, robot harus kembali ke kondisi awal (semua ruangan yang dibuka harus tertutup, jumlah "(" sama dengan jumlah ")")

Ditanya : Apakah rangkaian tersebut valid (YES / NO)?

Jawaban : Gampangnya kita pake satu angka penghitung aja yang nunjukkin "robot ini sekarang lagi di kedalaman berapa ruangan". sebut aja counter.

- > ketemu "(" itu counter nambah 1 (masuk ruangan baru, makin dalem)
- > ketemu ")" itu counter berkurang 1 (keluar satu ruangan)

Nah ada 2 hal yang bikin gagal

- > Kalau di tengah jalan counternya sampe minus, berarti dia nyoba keluar padahal lagi gak di dalam ruangan manapun. Ini langsung gagal, gak usah lanjut-lanjut lagi.
- > Kalau di akhir cerita counternya bukan 0, berarti masih ada ruangan yang ngegantung, belum ditutup.

Kalau dari awal sampe akhir counternya gak pernah minus, dan di akhir pas balik ke 0, baru deh valid.

Algoritmanya

1. Inisialisasi counter = 0
2. Baca karakter satu-satu dari kiri ke kanan
 - > ($\rightarrow$ counter + 1
 - >) $\rightarrow$ counter - 1
 - > Kalau pas dicek counter udah minus, langsung vonis NO, ga perlu lanjut baca sisanya.

3. Kalau sampe habis dan ga pernah minus,
cek counter == 0
> Iya → YES
> engga → NO.